

# JUEGO OTHELLO EN CLIPS

Jon Rabadán de la Vega, Jon Barrutia de Juan, Lara García Ogueta

## ÍNDICE

|          |                                            |   |
|----------|--------------------------------------------|---|
| 1.       | Introducción .....                         | 3 |
| 2.       | Diseño del sistema .....                   | 3 |
| 2.1.     | Representación de la información .....     | 3 |
| 2.2.     | Fases del sistema .....                    | 3 |
| 2.3.     | Reglas del sistema .....                   | 4 |
| 2.3.1.   | Fase pedir-tamaño .....                    | 4 |
| 2.3.2.   | Fase inicializar-tablero .....             | 4 |
| 2.3.3.   | Fase calcular-movimientos .....            | 4 |
| 2.3.4.   | Fase imprimir-tablero .....                | 4 |
| 2.3.5.   | Fase esperar-jugada .....                  | 5 |
| 2.3.5.1. | Regla pedir-jugada .....                   | 5 |
| 2.3.5.2. | Regla turno-máquina .....                  | 5 |
| 2.3.6.   | Fase verificar-jugada .....                | 5 |
| 2.3.6.1. | Regla validar-jugada-correcta .....        | 5 |
| 2.3.6.2. | Regla validar-jugada-incorrecta .....      | 6 |
| 2.3.7.   | Fase voltear-fichas .....                  | 6 |
| 2.3.8.   | Fase limpiar-movimientos .....             | 6 |
| 2.3.9.   | Control de turno y fin de partida .....    | 6 |
| 2.3.9.1. | Regla pasar-turno-sin-movimientos .....    | 6 |
| 2.3.9.2. | Regla detectar-fin-por-tablero-lleno ..... | 7 |
| 2.3.9.3. | Regla detectar-fin-por-doble-pase .....    | 7 |
| 2.3.10.  | Fase fin-juego .....                       | 7 |
| 2.4.     | Funciones .....                            | 7 |
| 2.5.     | Heurísticos .....                          | 7 |
| 3.       | Herramientas de ayuda utilizadas .....     | 8 |
| 4.       | Conclusiones .....                         | 8 |

## 1. INTRODUCCIÓN

El objetivo de este proyecto es desarrollar un sistema inteligente capaz de jugar al juego Othello contro una persona usuaria, utilizando el sistema basado en reglas CLIPS.

Othello es un juego de estrategia en el que dos jugadores colocan fichas en un tablero con el objetivo de capturar fichas del oponente. Basándonos en el tablero clásico o estándar (8x8), cada participante comienza con 32 fichas, que se irán colocando en el tablero. En el inicio del juego, se colocan 4 fichas en el centro del tablero y con las 30 restantes se juega. El jugador que al terminar la partida tenga el mayor número de fichas propias gana.

Además, también es posible jugar en un tablero de distinta dimensión, aunque recomendamos el clásico 8x8 para una experiencia óptima y fiel a los valores tradicionales.

## 2. DISEÑO

### 2.1. Representación de la información

La representación del problema se ha realizado mediante hechos (facts) y plantillas (deftemplate) en CLIPS. Cada plantilla representa una parte concreta del estado del juego y permite que las reglas trabajen sobre ella de manera estructurada

Estructuras principales:

#### - *juego*

Representa el estado global del sistema. Contiene dos atributos:

- **tamano**: dimensión del tablero (4x4, 6x6, 8x8, etc.).
- **fase**: indica en qué etapa del programa se encuentra el sistema.

El atributo **fase** puede tomar los siguientes valores:

- pedir-tamano
- inicializar-tablero
- calcular-movimientos
- imprimir-tablero
- esperar-jugada
- verificar-jugada
- voltear-fichas
- limpiar-movimientos
- fin-juego

Esta plantilla funciona como una máquina de estados que controla el flujo completo del juego.

#### - *casilla*

Representa cada posición individual del tablero. Está formada por:

- **fila**: fila de la casilla.
- **columna**: columna de la casilla.

- **estado**: contenido actual de la casilla.

El atributo **estado** puede tomar tres valores:

- **vacía** → no hay ficha.
- **negra** → contiene una ficha negra.
- **blanca** → contiene una ficha blanca.

Cada casilla del tablero existe como un hecho independiente, lo que facilita consultar y modificar posiciones concretas mediante reglas.

#### - **partida**

Contiene la información dinámica de la partida actual. Sus atributos son:

- **turno**: jugador que debe jugar actualmente.
- **fichas-negras**: contador de fichas negras.
- **fichas-blancas**: contador de fichas blancas.
- **humano**: color asignado al jugador humano.
- **maquina**: color asignado a la máquina.
- **pases-consecutivos**: número de turnos consecutivos en los que no ha habido movimientos válidos.

El atributo **turno** puede valer:

- negra
- blanca

El contador pases-consecutivos se utiliza para detectar el final de la partida cuando ambos jugadores no pueden mover.

#### - **movimiento-valido**

Representa una jugada legal para el jugador actual. Incluye:

- **fila**: fila donde puede jugarse.
- **columna**: columna donde puede jugarse.
- **capturas**: número de fichas enemigas que serían capturadas.
- **score**: puntuación heurística total del movimiento.

Estos hechos son temporales y se recalculan en cada turno dependiendo del estado actual del tablero.

#### - **jugada**

Representa la acción elegida por un jugador. Contiene:

- **fila**
- **columna**

Puede ser generado tanto por el jugador humano como por la inteligencia artificial.

- ***movimiento***

Representa un movimiento realizado durante la partida. Incluye:

- **fila**
- **columna**
- **jugador**
- **fichas-capturadas**

Aunque no participa directamente en el flujo principal del juego, permite almacenar información detallada de movimientos realizados.

- ***voltear-direccion***

Representa una dirección concreta en la que pueden capturarse fichas después de una jugada. Contiene:

- **fila-origen**
- **columna-origen**
- **dx**
- **dy**
- **jugador**

Los atributos **dx** y **dy** representan el desplazamiento de una de las ocho direcciones posibles del tablero. Estos hechos temporales permiten procesar el volteo de fichas dirección por dirección.

- ***Vector global de direcciones***

Además de las plantillas, se utiliza una variable global:

- **?dirs**

Esta variable almacena las ocho direcciones posibles del tablero:

- norte
- noreste
- este
- sureste
- sur
- suroeste
- oeste
- noroeste

Gracias a este vector, el sistema puede recorrer todas las direcciones utilizando la misma lógica sin duplicar código.

## 2.2. Fases del sistema

Para poder controlar correctamente la ejecución del programa, se ha diseñado un mecanismo de control por fases. Para ello, se ha utilizado el hecho *juego*, que contiene un atributo *fase* que indica en que punto del programa se encuentra el sistema. Cada fase representa una etapa concreta del juego.

De esta forma, cada regla solo se activa si la fase coincide con su condición y al finalizar modifica la fase para continuar con el flujo del programa. Las fases son las siguientes:

- 2.2.1. Fase *pedir-tamano*:
- 2.2.2. Fase *inicializar-tablero*
- 2.2.3. Fase *calcular-movimientos*
- 2.2.4. Fase *imprimir tablero*
- 2.2.5. Fase *esperar-jugada*
- 2.2.6. Fase *verificar-jugada*
- 2.2.7. Fase *voltear-fichas*
- 2.2.8. Fase *limpiar-movimientos*
- 2.2.9. Fase *fin-juego*

## 2.3. Reglas

### 2.3.1. Fase *pedir-tamano*

En esta fase inicial se solicita el tamaño del tablero al usuario.

- Regla *pedir-tamano*:

Esta regla se activa en la fase *pedir-tamano* y es la encargada de inicializar el tamaño del tablero. La regla solicita al usuario un número y comprueba que sea entero, par y mayor o igual a cuatro.

Si el número introducido no es válido, solicita de nuevo el tamaño del tablero hasta obtener un tamaño válido.

Finalmente, cambia a la fase *inicializar-tablero*.

### 2.3.2. Fase *inicializar-tablero*

En esta fase se crea el estado inicial del juego

- Regla *inicializar-tablero*:

Crea todas las casillas del tablero en función del tamaño introducido iniciándolas como *vacía*. Posteriormente, coloca las fichas iniciales.

Además, crea el hecho *partida* estableciendo el turno inicial como *negra*. Finalmente, cambia a la fase *calcular-movimientos*.

### 2.3.3. Fase *calcular-movimientos*

En esta fase se determinan los movimientos legales de cada jugador actual.

- Regla *detectar-movimientos*:

Recorre todas las casillas vacías del tablero y analiza en las ocho direcciones posibles, si colocar una ficha en una posición permitiría capturar fichas del oponente.

Para cada movimiento válido calcula el número de fichas capturadas, aplica la heurística de puntuación y genera un hecho *movimiento-valido*.

- Regla *finalizar-detección*

Se ejecuta cuando termina el cálculo de movimientos, esto lo hemos controlado utilizando *salience*. Hemos dado una prioridad baja a la regla con *salience -10* para asegurarnos de que se ejecuta únicamente cuando todas las posibles instancias de la regla *detectar-movimientos* han sido procesadas. Su función es cambiar la fase a *imprimir-tablero*, asegurando que todos los movimientos han sido previamente calculados.

#### 2.3.4. Fase *imprimir-tablero*

En esta fase se muestra el estado actual del juego de forma visual al usuario.

- Regla *imprimir-tablero*:

Esta regla se encarga de representar el tablero completo por pantalla en cada turno. Para ello, recorre todas las casillas del tablero y muestra su contenido en función de su estado:

- “N” para fichas negras
- “B” para fichas blancas
- “.” para casillas vacías
- “o” para movimientos válidos

Los movimientos válidos no forman parte del tablero (casillas) directamente, sino que están representados mediante hechos *movimiento-valido*. Durante la impresión, la regla consulta estos hechos y si una casilla vacía coincide con uno de ellos la marca con el símbolo “o” para indicar que es una jugada posible.

Además la regla calcula el número total de fichas negras y blancas recorriendo todas las casillas y muestra la puntuación actual de los dos jugadores. También imprime el turno activo.

Una vez mostrado el tablero, el sistema pasa a la fase *esperar-jugada*.

#### 2.3.5. Fase *esperar-jugada*

En esta fase se gestiona la interacción con el jugador humano o la máquina.

- Regla *pedir-jugada*:

Se activa si es el turno del jugador humano. Solicita fila y columna y genera un hecho *jugada* con los valores introducidos. Trás la ejecución el sistema pasa a la fase *verificar-jugada*.

- Regla *turno-maquina*:  
Se activa si es el turno de la máquina. Recorre todos los hechos *movimiento-valido* y selecciona el que tenga mayor puntuación heurística. Después, genera un hecho *jugada* con la mejor opción. Tras la ejecución el sistema pasa a la fase *verificar-jugada*.

#### 2.3.6. Fase *verificar-jugada*

En esta fase se comprueba la validez de la jugada.

- Regla *validar-jugada-correcta*:  
Se activa cuando la jugada coincide con un movimiento válido. En este caso, se coloca la ficha en el tablero y se generan los hechos *voltear-direccion* para cada dirección posible. Además, se cambia el turno al jugador contrario, se reinicia el contador de pases y finalmente, se cambia a la fase *voltear-fichas*.
- Regla *validar-jugada-incorrecta*:  
Se activa cuando la jugada no es válida. Muestra un mensaje de error, elimina la jugada introducida eliminando el hecho *jugada* y cambia el estado del sistema a la fase *esperar-jugada* para que el jugador introduzca una jugada de nuevo.

#### 2.3.7. Fase *voltear-fichas*

En esta fase se ejecuta la captura de fichas

- Regla *voltear-fichas*:  
Procesa cada hecho *voltear-direccion*. Esta regla tiene una prioridad alta, *salience 10*, para asegurarnos de que el volteo de fichas se realiza inmediatamente después de que se generen los hechos *voltear-direccion*. Para cada dirección comprueba si existe una secuencia válida de fichas enemigas encerradas y si es así, cambia su estado al del jugador actual. Esta regla se ejecuta tantas veces como direcciones existen.
- Regla *fin-volteo*:  
Se activa cuando ya no quedan hechos *voltear-direccion*. Su función es indicar que el proceso de captura ha terminado y cambiar la fase a *limpiar-movimientos*.

#### 2.3.8. Fase *limpiar-movimientos*

En esta fase se prepara el sistema para el siguiente turno

- Regla *limpiar-movimientos*:



Elimina todos los hechos *movimiento-valido* generados en el turno anterior para evitar inconsistencias.

Una vez se han eliminado, el sistema vuelve a la fase *calcular-movimientos*, iniciando un nuevo ciclo.

Esta regla es muy importante, porque los movimientos válidos dependen del estado actual del tablero y se tienen que recalcular en cada turno.

#### 2.3.9. Control de turno y fin de partida

Estas reglas pueden activarse en distintas fases para gestionar distintas situaciones

- Regla *pasar-turno-sin-movimientos*:

Se activa cuando no existen movimientos válidos para el jugador actual. Se ha utilizado *salience 5* para dar prioridad a la comprobación de ausencia de movimientos antes de seguir con el flujo normal del juego. En este caso, se informa al jugador y se cambia el turno. También se incrementa el contador de pases consecutivos y se vuelven a calcular los movimientos posibles para el otro jugador.

- Regla *detectar-fin-por-tablero-lleno*:

Se activa cuando no quedan casillas vacías en el tablero. Esta regla tiene una prioridad muy alta, *salience 50*, para detectar cuanto antes la condición de fin de partida y evitar ejecuciones innecesarias. Cambia directamente a la fase *fin-juego*.

- Regla *detectar-fin-por-doble-pase*:

Se activa cuando ambos jugadores han pasado turno consecutivamente. Finaliza la partida.

#### 2.3.10. Fase *fin-juego*

Fase final del sistema.

- Regla *mostrar-ganador*:

Cuenta el número de fichas de cada jugador e indica quién es el ganador, en caso de que exista o empate.

Muestra el resultado por pantalla y detiene la ejecución del programa.

### 2.4. Funciones

En este proyecto no se han definido funciones propias, ya que toda la lógica del sistema se ha implementado mediante reglas.

Hemos utilizado funciones ya definidas del lenguaje como:

- *loop-for-count* para recorrer el tablero
- *do-for-fact* y *do-for-all-facts* para consultar hechos
- *bind* para manejar variables
- *while* para iterar

Además hemos utilizado un vector global de direcciones *\*dirs\** para analizar las ocho direcciones posibles en el tablero.

## 2.5. Heurísticos

Implementados como “prioridad”, sirven como añadido al criterio de elección, añadiendo o quitando valor para la elección en el turno de la máquina. Por lo que la elección se basa en la búsqueda voraz (cuántas fichas se capturan con cada posibilidad) sumada a los puntos opcionales siguientes:

### 2.5.1. Esquinas (+100 pts)

Las casillas de las esquinas tienen la máxima prioridad, ya que son posiciones que no pueden ser capturadas por el oponente. Por eso, si una jugada permite ocupar una esquina, se le asigna una puntuación muy alta.

### 2.5.2. Adyacentes a esquinas (-50 pts)

Estas posiciones son peligrosas, ya que en muchos casos permiten al oponente capturar la esquina en el siguiente turno. Por este motivo, se penalizan mucho restando puntuación.

### 2.5.3. Bordes (+10 pts)

Las casillas situadas en los bordes del tablero son relativamente seguras y útiles estratégicamente, por eso son importantes aunque no tanto como las esquinas. Así que le sumamos algo a la puntuación.

## 3. HERRAMIENTAS DE AYUDA

Durante el ciclo de vida del proyecto, se ha hecho uso de diversas herramientas para comprender el dominio del problema y agilizar el desarrollo técnico:

- **Investigación y Dominio del Juego:** Al inicio del proyecto, fue necesario realizar un "trabajo de campo" para interiorizar las mecánicas y la visión espacial de Othello/Reversi. Para ello, consultamos la documentación de [Wikipedia](#) y utilizamos plataformas online interactivas (como <https://cardgames.io/es/reversi/>) y aplicaciones móviles (Offline Games).
- **Estudio de Heurísticas:** Para poder dotar de inteligencia artificial a nuestro agente, visualizamos material audiovisual didáctico en YouTube (específicamente el vídeo "[How to win at Othello](#)"). Esto fue clave para entender la importancia táctica de las esquinas y los bordes, conceptos que posteriormente trasladamos a nuestro sistema de puntuación.
- **Soporte de Inteligencia Artificial (Gemini Pro y VSCode Copilot):** Se utilizaron asistentes de programación basados en IA como apoyo en tres frentes principales:
  - **Estructuración y Arquitectura:** Para diseñar de forma limpia la máquina de estados finitos que controla las fases del juego.

- **Sintaxis Específica de CLIPS:** Como guía de referencia adicional para comprender y aplicar correctamente funciones avanzadas y complejas del lenguaje declarativo (como *do-for-fact*, *loop-for-count* y el manejo de listas para la lógica vectorial de capturas).
- **Interfaz de Usuario (UI):** Para generar el "Arte ASCII" utilizado en los mensajes finales de victoria o derrota, mejorando así la presentación visual exigida.

#### 4. CONCLUSIONES

Tras terminar el proyecto y hacer varias pruebas jugando, hemos sacado las siguientes conclusiones:

- **Adaptación a CLIPS:** Al principio nos costó bastante cambiar el "chip" a la programación basada en reglas, ya que es muy distinto a lo que estamos acostumbrados. Sin embargo, organizar el código por fases nos salvó de muchos problemas y evitó que las reglas se ejecutaran cuando no tocaba.
- **Fichas totales:** Hemos asumido que el número total de fichas no es estricto a 32, sino que varía lógicamente según el tamaño del tablero. Así, las fichas por jugador son  $\frac{n^2}{2}$ , siendo 'n' la dimensión del tablero. Por ejemplo, en el modo clásico (8x8), la fórmula nos da exactamente las 32 fichas por jugador, pero permite escalar el juego correctamente a cualquier otra dimensión sin desbordar el tablero.
- **Nuestra IA es (casi) invencible:** Aunque los heurísticos no son extremadamente complejos, la máquina juega sorprendentemente bien. De hecho, probando el juego, no hemos conseguido ganarle.
- **Curiosidad en tableros pequeños:** El juego escala bien a tableros grandes asumiendo el número proporcional de fichas. Pero probando el tamaño mínimo (4x4), nos dimos cuenta de que quien empieza tiene muchísima desventaja: al haber tan poco espacio, te ves obligado a colocar fichas en casillas penalizadas casi desde el primer turno.
- **Posibles mejoras (Minimax):** Nos llegamos a plantear implementar el algoritmo Minimax para que la IA calculara sus jugadas basándose en nuestros posibles movimientos futuros. Al final lo descartamos porque el árbol de posibilidades sería enorme y complicaría demasiado el código para los requisitos actuales, pero sería el paso lógico si quisiéramos mejorar el proyecto en el futuro. De todos modos, la recompensa por implementar Minimax podría no ser proporcional a su dificultad, dado que nuestra IA ya es muy difícil de vencer.